



INTERNSHIP PROJECT BRIEF

Project LOOP

AI Customer-Feedback Intelligence Platform

Web Development Track · Corporate-Grade

4-Week Build · Next.js 14 + TypeScript + PostgreSQL + Claude AI

PROGRAM Zidio Internship	DOMAIN Web Development	DURATION 4 Weeks (1 Month)
DIFFICULTY Industry / Corporate	TEAM SIZE 1-4	PROJECT WEIGHT 50 / 100 Marks

Document Control

Field	Detail
Document title	Project LOOP — AI Customer-Feedback Intelligence Platform
Document type	Internship Project Brief & Specification
Version	1.0
Status	Released
Owner	Zidio Development — Web Development Engineering Team
Audience	Web Development Track interns (current cohort)
Classification	Internal / Confidential
Review cycle	Per cohort

REVISION HISTORY

Version	Date	Author	Summary of change
1.0	On issue	Zidio Development Engineering Team	Initial release of the brief for the current cohort.

How to use this document

Read Sections 01–04 first to understand what you are building and why. Then work week-by-week through Section 10 (Timeline) using Section 08 (Feature Specification) as your build checklist.

Everything you are graded on is in Section 12 (Evaluation). Everything you must hand in is in Section 13 (Submission). Do not skip these.

Contents

Document Control.....	2
01 Project Overview.....	4
02 Business Context & Problem Statement.....	5
03 Objectives & Learning Outcomes	6
04 Project Scope	6
05 Technology Stack	7
06 System Architecture.....	8
07 Data Model	9
08 Feature Specification	10
09 AI Implementation Notes.....	12
10 Project Timeline — 4-Week Sprint Plan.....	13
11 Milestones & Deliverables	15
12 Evaluation & Scoring Rubric.....	16
13 Submission Requirements	18
14 Getting Started	18
15 Coding Standards & Git Workflow	19
16 Suggested Repository Structure.....	19
17 Support, Mentorship & Escalation	20
18 Originality & Academic Integrity.....	20
19 Appendix A — Sample Feedback Data	21
20 Appendix B — Resources	21
21 Appendix C — Glossary.....	22

01 Project Overview

Project LOOP is a corporate-grade web application that helps companies make sense of the customer feedback they receive every day. Support tickets, app-store reviews, survey responses, and sales notes pile up faster than any team can read them. LOOP ingests that feedback, uses AI to classify and cluster it, surfaces what is trending, and answers plain-English questions about what customers actually want.

You are building the kind of internal product that real SaaS companies pay for — comparable in spirit to tools like Interpret, Dovetail, or Productboard Insights. It is deliberately not a to-do app or a basic blog with a chatbot bolted on. It forces you to handle the things you would actually own as a junior engineer at a product company: multi-tenant data, role-based access, a clean API layer, a real database, a charts dashboard, and AI features that require genuine understanding to implement.

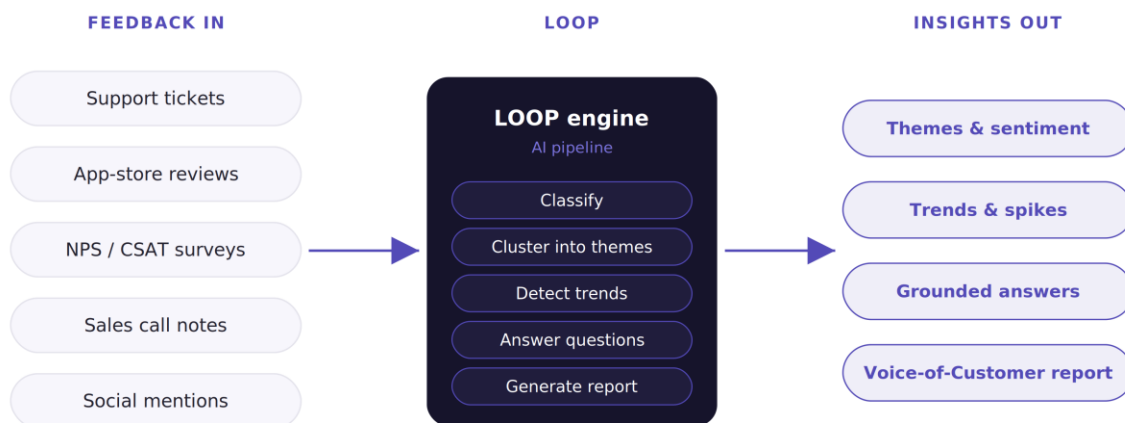


Figure 1. How LOOP turns scattered customer feedback into ranked, evidence-backed decisions.

Why this project was chosen for you

Portfolio value — “I built a multi-tenant AI feedback-intelligence platform” is a sentence that gets interviews. It demonstrates full-stack ownership, not a tutorial clone.

Hard to fake — tenant isolation, RBAC, and retrieval-grounded AI cannot be copy-pasted from a single YouTube video. Your understanding will show in the build.

Measurable — the feature set maps cleanly onto Zidio’s scoring rubric, so your grade reflects what you shipped, not opinion.

1.1 At a glance

Item	Detail
Product name	LOOP — “Close the loop on customer feedback.”
What it does	Ingests multi-channel customer feedback and turns it into themes, sentiment, trends, and answers.

Item	Detail
Who uses it	Product managers, support leads, and founders at a company (the “tenant”).
Core engineering challenge	Multi-tenant data isolation + role-based access + a clean, documented API.
AI challenge	Structured classification, theme clustering, retrieval-grounded Q&A, and report generation.
Primary deliverable	A deployed, working web app + source repository + documentation + demo video.

02 Business Context & Problem Statement

2.1 The problem

Every product company is drowning in customer feedback that arrives through too many doors at once. A typical mid-size SaaS company might receive feedback through all of the following in a single week:

- Support tickets and live-chat transcripts.
- App-store and review-site ratings with written comments.
- NPS and CSAT survey free-text responses.
- Sales and success call notes (“the customer asked for X again”).
- Social mentions and community posts.

Individually each item is a sentence or two. Collectively they hold the answer to the most valuable question a company can ask — “what should we build, fix, or improve next?” — but no human has time to read, tag, and synthesise hundreds of items a week. So the feedback rots in spreadsheets and inboxes, and product decisions get made on gut feel and the loudest voice in the room.

2.2 The opportunity

LOOP closes that gap. It is the single place a team drops all of its feedback, and the AI does the reading: it tags every item, groups similar items into themes, flags what is spiking this week, and lets anyone ask questions in plain English and get answers grounded in the actual feedback. The team goes from “we think customers want this” to “43 customers asked for this in the last 30 days, and complaints about it are up 60% week-over-week.”

Your business framing for the demo

When you present LOOP, frame it as a product, not a class assignment. Imagine you are pitching it to a Head of Product who has 90 seconds. The headline is: “LOOP turns scattered customer feedback into a ranked, evidence-backed list of what to do next.” Build and demo with that sentence in mind.

03 Objectives & Learning Outcomes

3.1 Project objectives

1. Design and ship a multi-tenant web application where each company's data is fully isolated from every other company's.
2. Implement secure authentication and role-based access control across at least three roles.
3. Build a clean, predictable REST/Route-Handler API that the frontend consumes — no business logic leaking into UI components.
4. Integrate the Claude AI API to deliver at least three meaningful AI features that genuinely require AI (not cosmetic).
5. Deploy a production build to a public URL and document it so a stranger could run it locally.

3.2 Learning outcomes

By the end of this project you will be able to demonstrate, with evidence in your repo, that you can:

Skill area	What you will have proven
Data modelling	Designing a relational schema with tenancy, relationships, and constraints (Prisma + PostgreSQL).
Backend / API	Writing typed, validated, paginated API endpoints with proper error handling and auth guards.
Auth & security	Session handling, RBAC, and preventing cross-tenant data access.
AI engineering	Prompt design for structured output, retrieval-grounded answers, and summarisation.
Frontend	Dashboards, data tables, filters, charts, loading/empty/error states, responsive layout.
DevOps basics	Environment configuration, database migrations, and deploying to Vercel.
Professional habits	Git hygiene, documentation, and presenting work as a product.

04 Project Scope

Scope is fixed so that everyone is graded on a level field and so the project is finishable in four weeks. Build what is in scope well before adding anything beyond it.

4.1 In scope (required)

- Multi-tenant workspaces with three roles (Admin, Analyst, Viewer).
- Feedback ingestion: manual single entry, CSV bulk upload, and at least one simulated "channel" source.
- A feedback inbox with search, filtering, pagination, and a status workflow.
- An analytics dashboard with at least three charts driven by real data.

- Four AI features (Section 08): auto-classification, theme clustering & trends, Ask LOOP Q&A, and the Voice-of-Customer report.
- A public deployment, a README, and a demo video.

4.2 Out of scope (do not build)

These are explicitly excluded. They are not extra credit — building them at the expense of the required scope will lower your score, not raise it.

- Real third-party integrations (live Zendesk / App Store / Twitter pulls). Simulate channels with seed data instead.
- Billing, payments, or subscription tiers.
- Native mobile apps.
- Real-time collaboration (websockets / live cursors).
- Email/SMS delivery infrastructure.

If you finish early

Polish and harden first: tests, error states, accessibility, performance, and a genuinely impressive demo. Only after the required scope is excellent should you consider one stretch feature (see Section 11). A flawless core beats a half-working pile of extras every time.

05 Technology Stack

The stack is standardised so mentors can support everyone and so your work matches what Zidio builds in production. You may not substitute the core framework or database, but you may add libraries within the ecosystem.

Layer	Technology	Why
Framework	Next.js 14 (App Router) + TypeScript	Full-stack in one codebase; matches Zidio production.
Styling	Tailwind CSS	Fast, consistent, industry-standard utility CSS.
Database	PostgreSQL (Neon or Supabase free tier)	Relational integrity for multi-tenant data.
ORM	Prisma	Type-safe schema, migrations, and queries.
Auth	NextAuth (Auth.js)	Sessions, providers, and role handling.
AI	Anthropic Claude API (claude-sonnet-4-6)	Classification, summarisation, and Q&A.
Embeddings / search	pgvector or a hosted embeddings provider	Powers “Ask LOOP” retrieval.
Charts	Recharts	Dashboard visualisations.
Validation	Zod	Runtime validation on every API boundary.
Deployment	Vercel + hosted Postgres	One-command production deploys.

Java Full-Stack interns

Approved alternative stack: Spring Boot 3 + Java 17 + Spring Security + JPA/Hibernate + PostgreSQL for the backend, with a React (Vite) frontend, deployed via Render/Railway + Vercel. Every requirement, milestone, and rubric line in this document applies identically — only the implementation technology changes. Confirm your stack with your mentor before Day 2.

06 System Architecture

LOOP follows a standard three-tier architecture. The browser talks only to your own API layer; the API layer is the only thing that talks to the database and to the Claude API. Never call the AI provider directly from the browser — your API key must stay server-side.

6.1 Request flow

1. The browser (React Server/Client Components) renders the UI and calls your API route handlers.
2. Route handlers authenticate the session, check the user's role, and scope every query to the caller's workspace.
3. Prisma reads/writes PostgreSQL. All feedback rows carry a workspaceId; queries always filter on it.
4. For AI features, the route handler builds a prompt, calls the Claude API server-side, parses the response, and returns clean JSON to the browser.
5. For Ask LOOP, the handler first retrieves the most relevant feedback (vector search), then passes it to Claude as grounding context before answering.

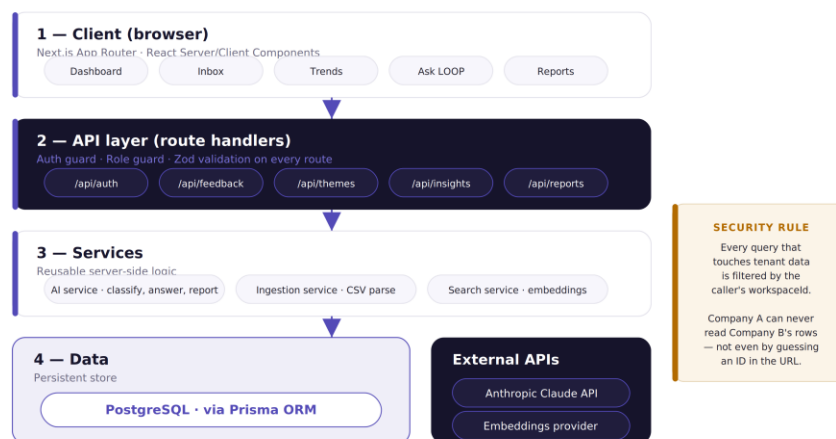


Figure 2. LOOP system architecture — a three-tier flow from browser to database, with every query scoped by workspace.

Non-negotiable security rule

Every single database query that touches feedback, themes, reports, or users MUST be filtered by the authenticated user’s workspaceId. A user from Company A must never be able to read a single row belonging to Company B — even by guessing an ID in the URL. Mentors will test this directly.

07 Data Model

Below is the minimum schema. You may add fields, but you must include these entities and relationships. Every tenant-owned table carries a workspaceId foreign key.

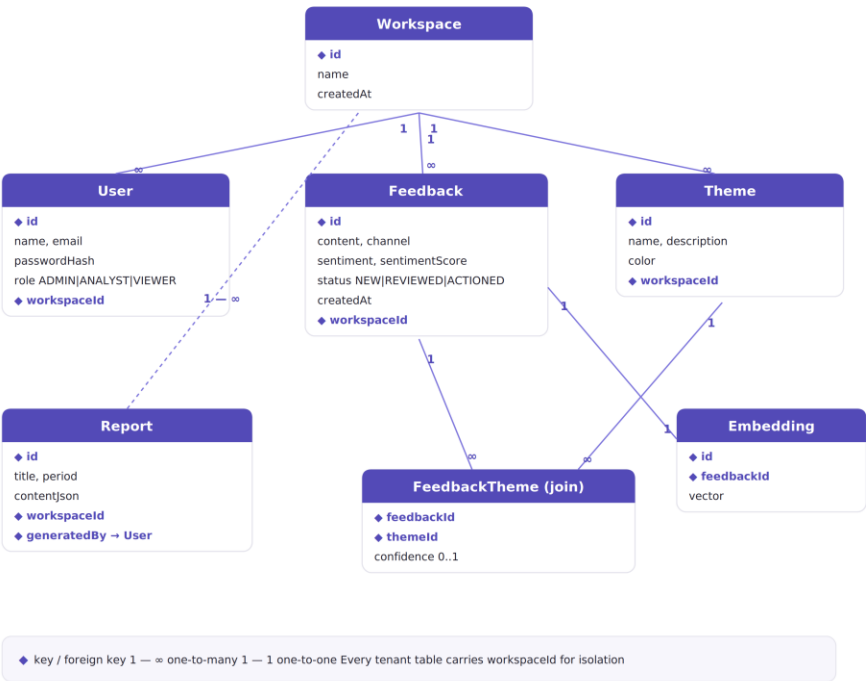


Figure 3. Entity-relationship schema — minimum entities, relationships, and tenant keys.

Entity	Key fields & relationships
Workspace	• id, name, createdAt • has many Users, Feedback, Themes, Reports
User	• id, name, email, passwordHash, role (ADMIN ANALYST VIEWER) • workspaceId → Workspace
Feedback	• id, content (text), channel, sourceRef, customerLabel • sentiment (POS NEU NEG), sentimentScore (-1..1) • status (NEW REVIEWED ACTIONED), createdAt • workspaceId → Workspace; has many FeedbackTheme; has one Embedding
Theme	• id, name, description, color • workspaceId → Workspace; has many FeedbackTheme

Entity	Key fields & relationships
FeedbackTheme	• feedbackId → Feedback, themeld → Theme (join table) • confidence (0..1)
Embedding	• id, feedbackId → Feedback, vector • used by Ask LOOP semantic search
Report	• id, title, periodStart, periodEnd, content.Json, createdAt • workspaceId → Workspace, generatedBy → User

Seed data is required

Ship a seed script that creates one demo workspace, three users (one per role), at least 120 realistic feedback items across several channels, and a handful of themes. Graders and you both need data to see the product work. A demo with three rows in it looks unfinished.

08 Feature Specification

Each feature below has a user story and acceptance criteria. Acceptance criteria are how mentors decide whether a feature “counts.” Treat them as your definition of done. Features are grouped into Core (the application) and AI (the intelligence).

8.1 Core features

C1 Authentication & workspaces

User story: *As a new user, I can create an account and a workspace, then log in securely so my company's data is private.*

Acceptance criteria

1. Sign-up creates a User and a Workspace; the creator becomes ADMIN.
2. Passwords are hashed; sessions persist across refresh.
3. Logged-out users are redirected away from protected pages.
4. All data the user sees is scoped to their workspace only.

C2 Role-based access control

User story: *As an admin, I can invite teammates and assign roles so people only do what their role permits.*

Acceptance criteria

1. Three roles exist: ADMIN, ANALYST, VIEWER.
2. Admins manage members and roles; Analysts ingest and manage feedback; Viewers are read-only.
3. The API enforces roles server-side — hiding a button is not enough.
4. Attempting a forbidden action returns 403, not a crash.

C3 Feedback ingestion

User story: *As an analyst, I can add feedback one item at a time, upload a CSV in bulk, or pull from a simulated channel.*

Acceptance criteria

1. Single-entry form with validation (content required, channel selected).
2. CSV upload parses rows and reports how many imported / how many failed.
3. At least one “channel” button seeds realistic items to simulate an integration.
4. Newly ingested items are queued for AI classification (Section 08 AI1).

C4 Feedback inbox

User story: *As an analyst, I can search, filter, and triage feedback so I can find and action what matters.*

Acceptance criteria

1. Server-side pagination (no loading 5,000 rows at once).
2. Filter by channel, sentiment, theme, status, and date range.
3. Full-text search over feedback content.
4. Status workflow: NEW → REVIEWED → ACTIONED, changeable inline.

C5 Analytics dashboard

User story: *As a product manager, I can see the shape of our feedback at a glance so I know where to focus.*

Acceptance criteria

1. At least three charts: volume over time, sentiment breakdown, and top themes.
2. Charts reflect the active filters / date range.
3. Key numbers shown as stat cards (total items, % negative, new this week).
4. Empty and loading states are handled gracefully.

8.2 AI features

These four features are the heart of LOOP and the heaviest-weighted part of the grade. Each must work end-to-end against real seeded data in your demo.

AI1 Auto-classification

User story: *As an analyst, every piece of feedback is automatically tagged so I never have to triage by hand.*

Acceptance criteria

1. On ingestion, each item is sent to Claude and returned with: sentiment, sentiment score, theme(s), and a short feature-area label.
2. Output is strictly structured (JSON) and validated before saving.
3. Classification is stored on the record — not recomputed on every page load.
4. A manual “re-classify” action exists for corrections.

AI2 Theme clustering & trends

User story: *As a product manager, I can see which themes are growing so I can react to emerging issues early.*

Acceptance criteria

1. Similar feedback is grouped into named themes with counts.
2. A trends view shows theme volume over time and flags themes spiking versus the previous period.
3. Clicking a theme drills into the underlying feedback items.
4. New feedback is assigned to existing themes where it fits, or forms a new one.

AI3 Ask LOOP (grounded Q&A)

User story: *As anyone on the team, I can ask plain-English questions and get answers backed by real feedback.*

Acceptance criteria

1. A chat-style box accepts questions like “What are users saying about onboarding?”
2. The system retrieves the most relevant feedback (semantic search) before answering.
3. Answers cite or list the specific feedback items they are based on.
4. The model must not invent feedback that is not in the data — grounding is mandatory.

AI4 Voice-of-Customer report

User story: *As a product manager, I can generate a weekly digest I could forward to leadership without editing.*

Acceptance criteria

1. One click generates a report for a chosen period.
2. The report summarises top themes, sentiment shifts, notable verbatim quotes, and recommended actions.
3. Reports are saved and viewable later, and exportable (PDF or shareable page).
4. Content is generated from the period’s actual data, not generic filler.

09 AI Implementation Notes

This section gives you a starting approach for each AI feature. You are expected to refine the prompts yourself — prompt design is part of the learning and part of the grade.

9.1 Structured classification (AI1)

Ask Claude to return JSON only, with a fixed schema, and validate it with Zod before saving. A robust approach:

- Send the feedback text plus your list of existing theme names so the model reuses themes instead of inventing new ones each time.
- Instruct the model to return only JSON: sentiment, sentimentScore (-1 to 1), themes (array), featureArea, and a one-line rationale.
- Strip any stray markdown fences, parse, validate, and fall back gracefully if parsing fails (retry once, then flag for manual review).

9.2 Retrieval-grounded Q&A (AI3)

Ask LOOP must answer from the data, not from the model’s imagination. The pattern is retrieve-then-answer:

1. Embed every feedback item when it is ingested; store the vector (pgvector or a vectors table).
2. On a question, embed the question, find the top-K most similar feedback items.
3. Pass those items to Claude as context with an instruction: answer only from the provided feedback; if the answer is not present, say so.
4. Return the answer plus the list of feedback items it used, so the user can verify it.

ASK LOOP — RETRIEVE, THEN ANSWER

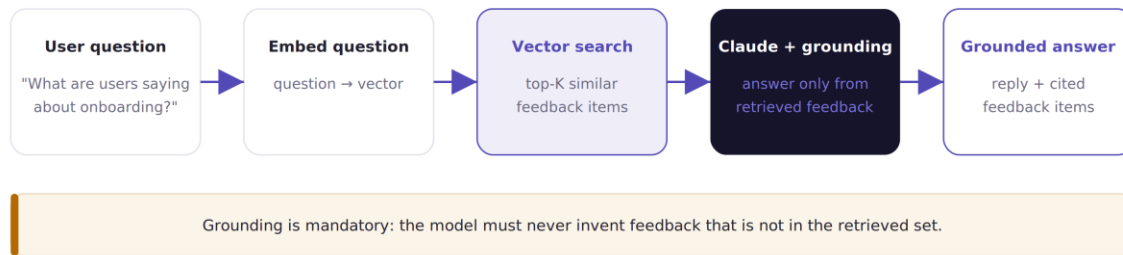


Figure 4. Ask LOOP retrieves the most relevant feedback first, then answers only from it.

9.3 Report generation (AI4)

Pre-compute the period's stats (top themes, counts, sentiment deltas, a few representative quotes) in code, then ask Claude to write the narrative around those numbers. This keeps the report accurate and cheap, and stops the model from hallucinating figures.

Cost, safety, and keys

Keep your API key server-side only. Never ship it to the browser or commit it to Git. Use environment variables.

Be economical. Classify on ingest and cache results; do not call the model on every page render. Batch where you can.

10 Project Timeline — 4-Week Sprint Plan

The project runs for four weeks (twenty working days, Monday to Friday). Each week is a sprint that ends in a demoable deliverable and a mentor checkpoint. Do not move to next week's work until the current week's deliverable is met — the build order is deliberate.

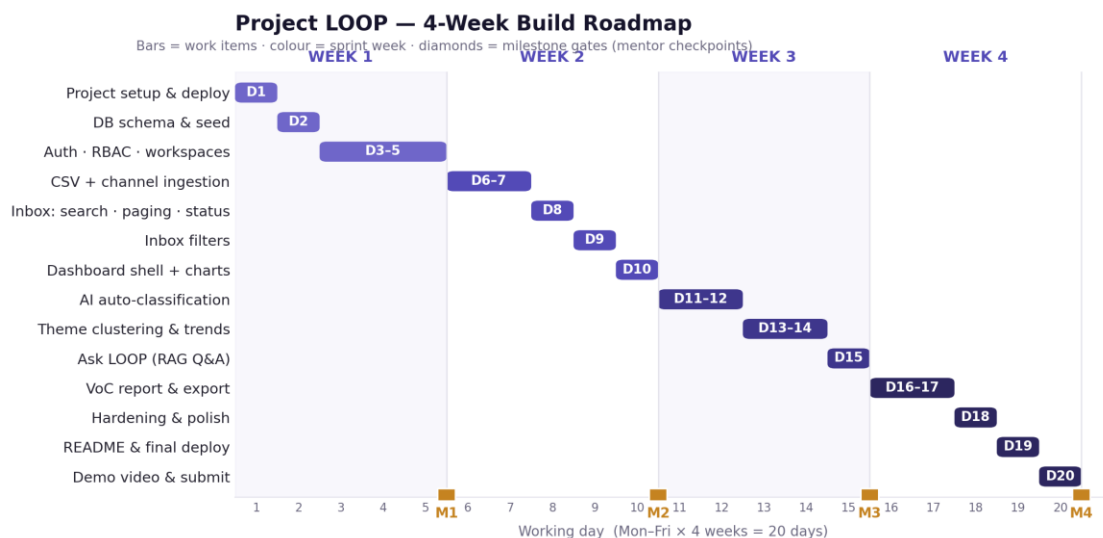


Figure 5. The four-week build roadmap: work items by sprint week, with milestone gates M1–M4.

WEEK

1

Foundation & Data Layer**End-of-week deliverable:** A deployed app where you can sign up, log in, and add & view feedback.

Day 1	Repo, Next.js + TypeScript + Tailwind setup, environment config, connect to PostgreSQL. Push “hello world” to Vercel so deployment works from day one.
Day 2	Design the Prisma schema (Section 07). Run the first migration. Write the seed script skeleton.
Day 3	Authentication with NextAuth: sign-up, login, logout, protected routes, session handling.
Day 4	Workspaces + RBAC roles. Ensure every query is scoped by workspaceId. Member list for admins.
Day 5	Single-entry feedback create + list (no AI yet). Deploy. Checkpoint with mentor.

WEEK

2

Core Application**End-of-week deliverable:** A working feedback-management app: bulk import, inbox with filters, dashboard shell.

Day 6	CSV bulk upload: parse, validate, import with a success/failure summary.
Day 7	Simulated channel source(s) that seed realistic feedback to mimic an integration.
Day 8	Feedback inbox: server-side pagination, search, and the status workflow.
Day 9	Inbox filters (channel, sentiment, theme, status, date range). Wire filters to the API.
Day 10	Dashboard shell with Recharts: volume, sentiment, top-themes charts (placeholder data ok). Deploy. Checkpoint.

WEEK

3

AI Integration**End-of-week deliverable:** Auto-classification, theme trends, and Ask LOOP all working against real data.

Day 11	AI service wiring: server-side Claude call, structured-JSON classification, Zod validation.
Day 12	Classify on ingest + store results; back-fill classification across seeded data; manual re-classify.
Day 13	Theme clustering: assign feedback to themes; theme list with counts and drill-down.
Day 14	Trends view: theme volume over time + spike detection vs previous period.
Day 15	Ask LOOP: embeddings on ingest, semantic retrieval, grounded answer with cited items. Deploy. Checkpoint.

WEEK

4

Intelligence & Production Polish**End-of-week deliverable:** A production-ready submission: VoC report, polished UX, README, and demo video.

Day 16	Voice-of-Customer report: pre-compute stats, generate narrative, save + view.
Day 17	Report export (PDF or shareable page). Connect real AI-driven numbers into the dashboard.
Day 18	Hardening: error handling, loading/empty states, 403/404 pages, responsive pass, accessibility basics.
Day 19	Write the README (setup, architecture, screenshots). Final seed data. Final deploy + smoke test.
Day 20	Record the 3–5 minute demo video. Final QA against the rubric. Submit before the deadline.

Buffer & realism

This plan is intentionally tight but achievable with focused daily effort. If you fall a day behind, protect the AI features (Week 3) and the deployment — those carry the most marks. Trim polish before you trim core functionality, and tell your mentor early if you are blocked.

11 Milestones & Deliverables

Four milestones gate the project. Each ends in a mentor checkpoint where you demo the deliverable. Hitting milestones on time is itself part of professional delivery.

Milestone	End of	Deliverable	Project pts	Status
M1 — Foundation	Week 1	Live URL with auth, RBAC, workspaces, and basic feedback CRUD.	10	☐
M2 — Core App	Week 2	Bulk import, inbox with filters & status, dashboard shell.	15	☐
M3 — AI Features	Week 3	Classification, theme trends, and Ask LOOP working end-to-end.	15	☐
M4 — Production	Week 4	VoC report, polished UX, README, demo video, final deploy.	10	☐

11.1 Optional stretch goals

Only attempt these once the entire required scope is excellent. They can strengthen a borderline submission but never substitute for core work.

- Saved views / segments in the inbox (e.g., “all negative onboarding feedback”).
- A “suggested actions” queue that turns recurring themes into draft tasks.
- Sentiment trend alerts (a banner when negativity spikes).
- Basic test coverage on the API layer (Vitest / Jest).

12 Evaluation & Scoring Rubric

Your project is scored within Zidio's standard 100-point internship rubric. This brief governs the Project (50) and Submission (20) components; Attendance (20) and Test (10) are assessed by the program separately.

12.1 Overall mark distribution

Component	Marks	How it is assessed here
Attendance	20	Program-level; daily presence and checkpoint participation.
Test / assessment	10	Program-level; track assessment.
Project	50	This brief, Sections 12.2–12.3 — milestones M1–M4.
Submission	20	This brief, Section 12.4 — deliverable quality.
Total	100	Combined internship score.

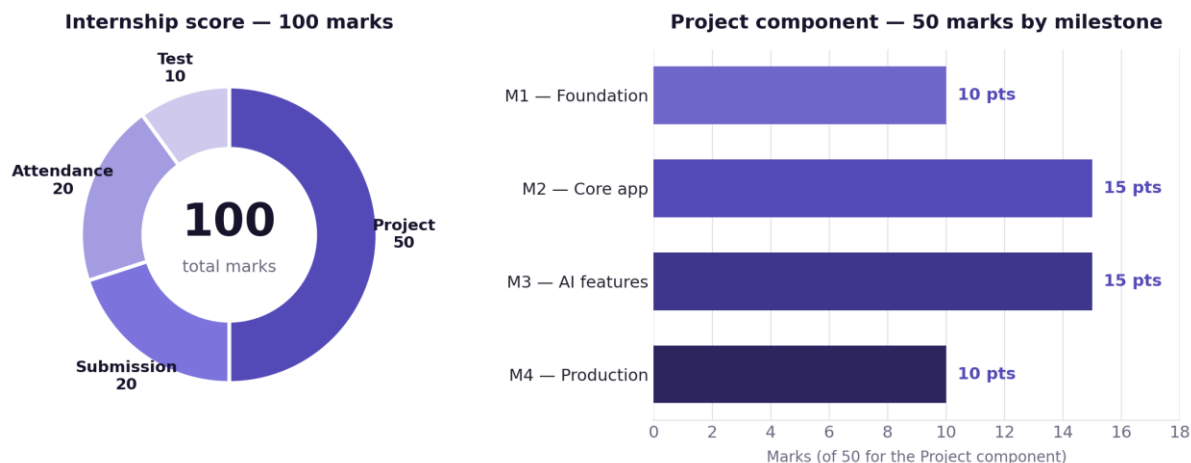


Figure 6. Mark distribution across the 100-point rubric, and the 50-point project broken down by milestone.

12.2 Project breakdown (50 marks)

Milestone	Marks	What earns full marks
M1 Foundation	10	Secure auth, three working roles, true workspace isolation, and a live deployment that actually loads.
M2 Core app	15	Bulk + single ingestion, a fast paginated inbox with all filters, working status workflow, and a real (not faked) dashboard.
M3 AI features	15	Classification, theme trends, and Ask LOOP each working on real data, with grounded (non-hallucinated) answers.
M4 Production	10	VoC report, polished states (loading/empty/error), responsive UI, and a clean README + demo.

12.3 Submission breakdown (20 marks)

Criterion	Marks	What earns full marks
Live deployment	5	A public URL that works, with seed data, that a grader can log into and explore.
README & docs	5	Clear setup steps a stranger can follow, architecture overview, and screenshots.
Code quality & Git	5	Readable, typed, organised code; meaningful commits; no secrets committed.
Demo video	5	A 3–5 minute walkthrough that tells the product story and shows every feature working.

Quality bar — what separates a top score

Average submission: features technically present but rough — broken edge cases, ugly empty states, AI that sometimes invents data.

Top submission: feels like a real product — fast, isolated per tenant, AI answers are grounded and useful, and the demo could be shown to a customer. Build for that bar.

13 Submission Requirements

Submit all of the following by the cohort deadline. A missing item is a missing mark — there is no partial credit for “almost submitted.”

1. GitHub repository link (public, or private with mentor access granted).
2. Live deployment URL (Vercel) with working seed data and login credentials for each role.
3. A README.md containing: what the project is, the tech stack, local setup steps, environment variables needed, the database/seed commands, an architecture summary, and screenshots.
4. A 3–5 minute demo video (unlisted YouTube / Google Drive link) walking through every feature.
5. The completed submission form for your cohort, with all links pasted in.
6. A 1-2 Minutes self-feedback video on internship.

Demo credentials checklist

Your README must list one set of login credentials per role (Admin / Analyst / Viewer) on the seeded demo workspace, so graders can verify RBAC without creating accounts. Never use a real password you reuse elsewhere.

14 Getting Started

Use this as your Day-1 setup path. Confirm exact package versions with your mentor; the steps below are the shape of the workflow.

14.1 Prerequisites

- Node.js 18 LTS or newer, and Git.
- A free PostgreSQL database (Neon or Supabase).
- An Anthropic API key (your mentor will advise how to obtain credits).
- A Vercel account connected to your GitHub.

14.2 First-run steps

```
# 1. Create the app and install dependencies
npx create-next-app@latest loop --typescript --tailwind --app
cd loop && npm install prisma @prisma/client next-auth zod recharts
npm install @anthropic-ai/sdk

# 2. Configure environment variables (never commit this file)
# .env -> DATABASE_URL, NEXTAUTH_SECRET, ANTHROPIC_API_KEY

# 3. Set up the database
npx prisma migrate dev --name init
npm run seed

# 4. Run locally, then deploy
npm run dev      # http://localhost:3000
vercel           # production deploy
```

Environment variables

Keep ANTHROPIC_API_KEY, DATABASE_URL, and NEXTAUTH_SECRET in .env locally and in Vercel's project settings for production. Add .env to .gitignore on commit one. A leaked key is a security incident and an automatic deduction.

15 Coding Standards & Git Workflow

You are graded partly on how you work, not just what you build. Treat this repo as if a teammate will inherit it.

15.1 Code quality

- TypeScript everywhere; avoid “any”. Let types document your data.
- Validate every API input with Zod; never trust client data.
- Keep business logic out of components — put it in API routes / service files.
- Handle errors explicitly; show users a friendly message, log the detail.
- Consistent formatting (Prettier) and linting (ESLint).

15.2 Git workflow

- Commit early and often with meaningful messages (e.g., “feat: add CSV ingestion with row validation”).

- Use a branch per feature and open a pull request into main, even solo — it builds the habit.
- Never commit secrets, .env files, or node_modules.
- Your commit history should tell the story of the four weeks — not one giant “final” commit at the end.

16 Suggested Repository Structure

A clean structure makes your project easy to grade and easy to extend. This is a recommended starting layout for the Next.js stack.

```
loop/  
  app/  
    (auth)/login, signup  
    (app)/dashboard, inbox, trends, ask, reports, settings  
  api/  
    auth/...      # NextAuth  
    feedback/     # CRUD + ingestion  
    themes/       # clustering + trends  
    insights/     # Ask LOOP Q&A  
    reports/      # VoC generation  
  components/     # UI building blocks (charts, tables, forms)  
  lib/  
    ai.ts         # Claude calls: classify, answer, report  
    search.ts     # embeddings + retrieval  
    auth.ts       # session + role guards  
    db.ts         # Prisma client  
  prisma/  
    schema.prisma  
    seed.ts  
  README.md  
  .env.example    # documented, no real secrets
```

17 Support, Mentorship & Escalation

You are not meant to do this alone. Zidio provides structured support — use it early rather than staying stuck.

Channel	Use it for
Weekly checkpoints	Demo your milestone, get feedback, and unblock direction. Mandatory.
Mentor office hours	Technical blockers, architecture questions, and code review.
Cohort channel	Peer help, sharing resources, and quick questions.
Escalation	If blocked for more than half a day, raise it in the cohort channel and tag your mentor — do not burn days silently.

The one rule of asking for help

Come with what you tried. “My CSV import throws this error, here’s the code and what I’ve already checked” gets a fast, useful answer. “It doesn’t work” does not. Showing your working is itself a graded professional skill.

18 Originality & Academic Integrity

This project is designed to be built, not copied. Originality is a condition of certification.

- You may use AI coding assistants and documentation — that is modern engineering. You must understand and be able to explain every line you submit.
- You may reference tutorials and starter templates, but the LOOP feature set, data model, and AI logic must be your own implementation.
- Directly copying another intern’s repository, or submitting a purchased/cloned project, results in disqualification.
- At your final review, mentors may ask you to explain or modify any part of your code live. Be ready to.

19 Appendix A — Sample Feedback Data

Use rows like these to build your CSV importer and seed script. Aim for at least 120 varied items across channels, sentiments, and topics so your AI features have something real to work on.

Channel	Sentiment	Example content
Support ticket	Negative	“Onboarding took forever — I couldn’t figure out how to invite my team.”
App store review	Positive	“The new dashboard is gorgeous and finally fast. Huge improvement.”
NPS survey	Neutral	“It does the job, but the mobile experience needs work.”
Sales call note	Negative	“Prospect wants SSO before they’ll sign — third time this month.”
Community post	Positive	“Love the new export feature, saved me an hour today.”
Support ticket	Negative	“Billing page keeps timing out when I try to download an invoice.”

Suggested CSV columns

content, channel, customer_label, created_at — with sentiment and themes left blank so your AI classifier fills them on import.

20 Appendix B — Resources

Official documentation for the core stack. Prefer primary docs over random tutorials.

Topic	Link
Next.js	https://nextjs.org/docs
Prisma	https://www.prisma.io/docs
NextAuth / Auth.js	https://authjs.dev
Anthropic Claude API	https://docs.claude.com
Recharts	https://recharts.org
Zod	https://zod.dev
Neon Postgres	https://neon.tech/docs
Vercel deploys	https://vercel.com/docs

21 Appendix C — Glossary

Term	Meaning
Multi-tenant	One application serving many isolated customers (“tenants”); each tenant’s data is private.
RBAC	Role-Based Access Control — permissions determined by a user’s role.
Tenant isolation	Guaranteeing one customer can never read or write another customer’s data.
Embedding	A numeric vector representing text meaning, used for semantic similarity search.
RAG	Retrieval-Augmented Generation — fetch relevant data first, then let the model answer from it.
Grounding	Forcing the AI to answer only from provided data, not from memory or invention.
VoC	Voice of Customer — the synthesised view of what customers are saying.
Seed data	Sample records loaded into the database so the app is usable and demoable immediately.